

一. 练习内容

1. 用 Verilog 完成三态门的设计，并用 Vivado 进行仿真。
2. 用 Verilog 完成 4 位加法器的设计，并使用 Vivado 进行仿真。

二. 源程序及仿真结果截图

1. 三态门设计

设计代码：

```
`timescale 1ns / 1ps

module tri_state_buffer (
    input wire in,
    input wire enable,
    output wire out
);

    assign out = (enable) ? in : 1'bz;

endmodule
```

测试代码：

```
`timescale 1ns / 1ps

module test;

    reg in;
    reg enable;
    wire out;

    san_tai_men uut (
        .in(in),
        .enable(enable),
        .out(out)
    );

endmodule
```

```

initial begin
    in = 0;
    enable = 0;
    #10;

    in = 1;
    enable = 0;
    #10;

    in = 1;
    enable = 1;
    #10;

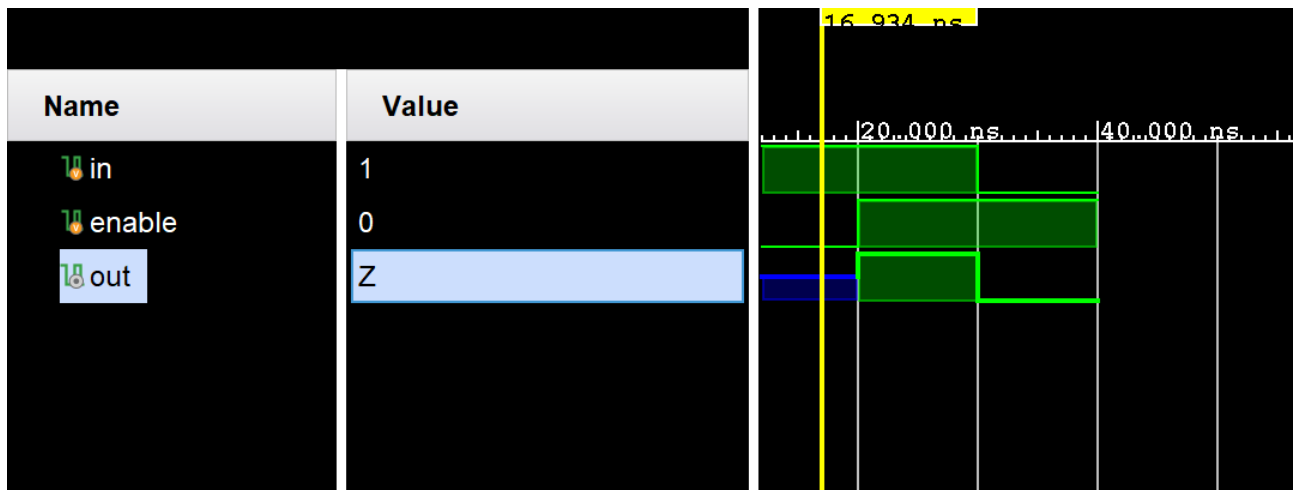
    in = 0;
    enable = 1;
    #10;

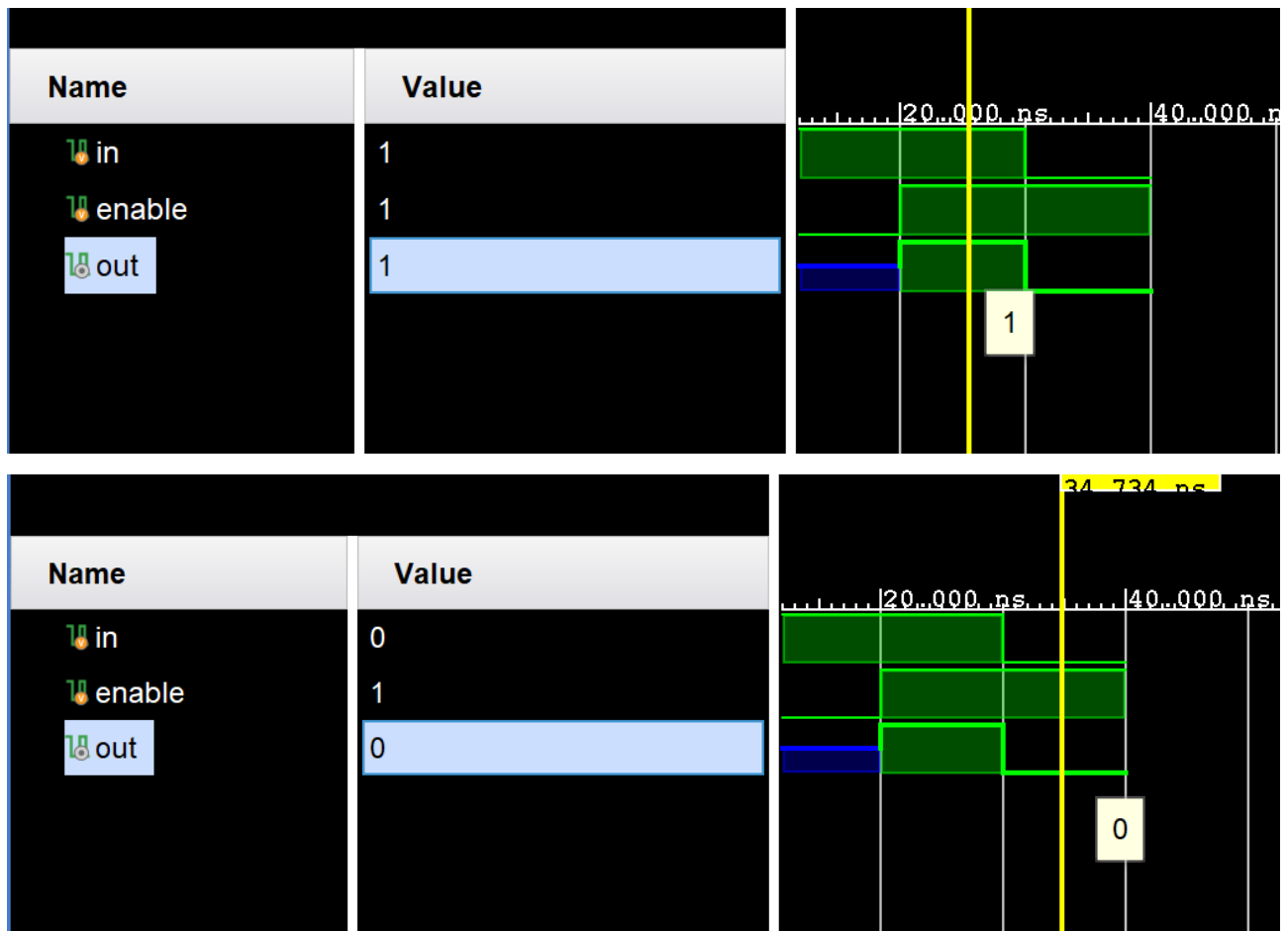
    $finish;
end

endmodule

```

仿真图：





2.4 位加法器的设计

设计代码:

```

`timescale 1ns / 1ps
module quan_jia_qi(
    cin,
    a,
    b,
    sum,
    cout
);
    input cin;
    input [3:0] a, b;

    output cout;
    output [3:0] sum;

    assign {cout, sum} = a + b + cin;
endmodule

```

测试代码:

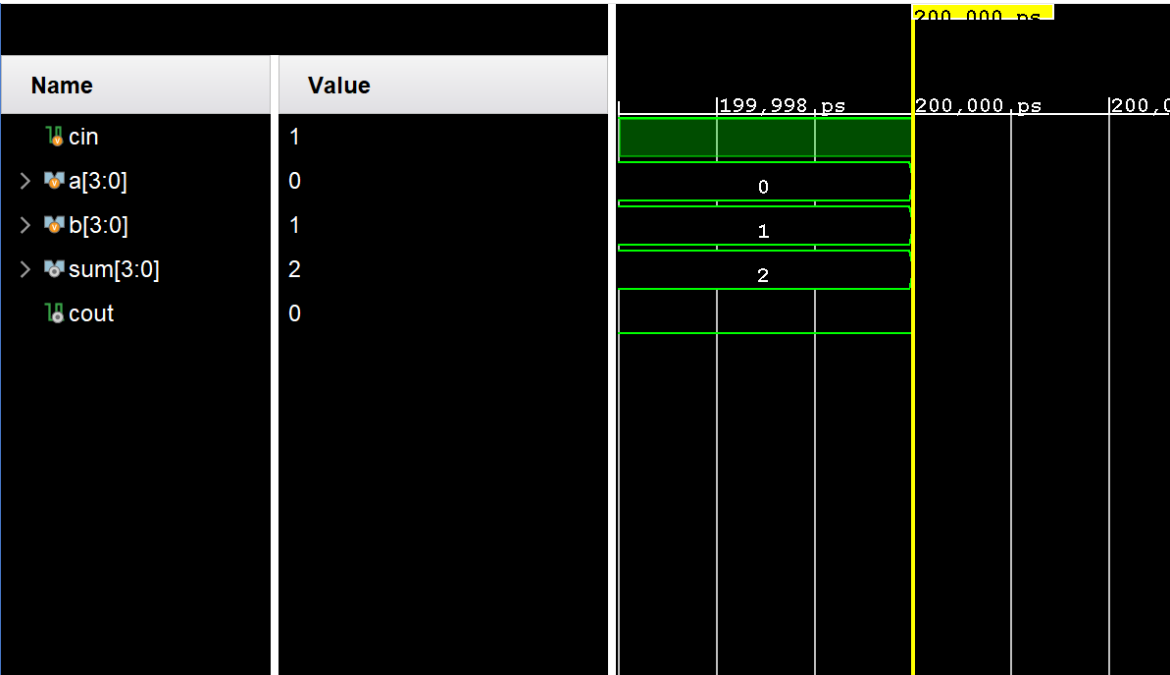
```
`timescale 1ns / 1ps
module test;

    reg cin;
    reg [3:0] a, b;
    wire [3:0] sum;
    wire cout;

    quan_jia_qi qjq(
        .a(a),
        .b(b),
        .cin(cin),
        .cout(cout),
        .sum(sum)
    );

    initial begin
        a = 0;
        b = 0;
        cin = 0;
        #100;
        a = 0;
        b = 1;
        cin = 1;
        #100;
        $finish;
    end
endmodule
```

仿真图：



一. 练习内容

1. 用 Verilog 完成 3-8 译码器的设计，并用 Vivado 进行仿真。
2. 用 Verilog 完成两个 4 位二进制数据比较器的设计，并使用 Vivado 进行仿真。

二. 源程序及仿真结果截图

1. 3-8 译码器的设计（无效输出低电平）:

设计代码:

```
`timescale 1ns / 1ps

module yi_ma_3_to_8 (
    input [2:0] a,
    input enable,
    output reg [7:0] y
);

    always @(*) begin
        if (enable) begin
            case (a)
                3'b000: y = 8'b0000_0001;
                3'b001: y = 8'b0000_0010;
                3'b010: y = 8'b0000_0100;
                3'b011: y = 8'b0000_1000;
                3'b100: y = 8'b0001_0000;
                3'b101: y = 8'b0010_0000;
                3'b110: y = 8'b0100_0000;
                3'b111: y = 8'b1000_0000;
                default: y = 8'b0000_0000;
            endcase
        end else begin
            y = 8'b0000_0000;
        end
    end
end

endmodule
```

测试代码：

```
`timescale 1ns / 1ps

module test;

    reg [2:0] a;
    reg enable;
    wire [7:0] y;

    yi_ma_3_to_8 uut (
        .a(a),
        .enable(enable),
        .y(y)
    );

    initial begin
        a = 3'b000;
        enable = 0;
        #10;

        a = 3'b000;
        enable = 1;
        #10;

        a = 3'b001;
        enable = 1;
        #10;

        a = 3'b010;
        enable = 1;
        #10;

        a = 3'b011;
        enable = 1;
        #10;

        a = 3'b100;
        enable = 1;
        #10;

        a = 3'b101;
        enable = 1;
```

```

#10;

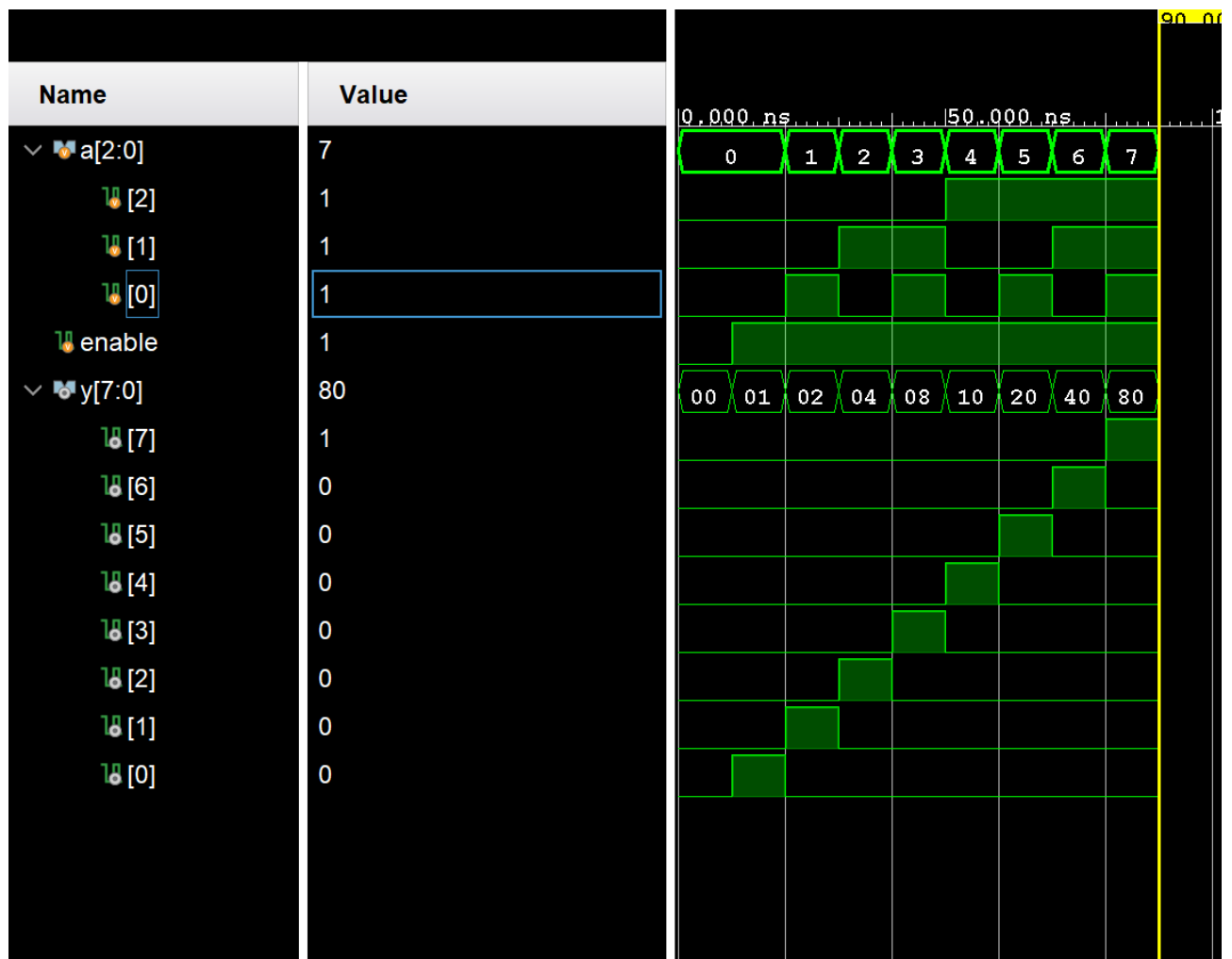
a = 3'b110;
enable = 1;
#10;

a = 3'b111;
enable = 1;
#10;

$finish;
end
endmodule

```

仿真图：



2. 四位二进制数据比较器

设计代码:

```
`timescale 1ns / 1ps

module binary_comparator (
    input [3:0] a,
    input [3:0] b,
    output reg greater,
    output reg equal,
    output reg less
);

always @(*) begin
    if (a > b) begin
        greater = 1;
        equal = 0;
        less = 0;
    end else if (a == b) begin
        greater = 0;
        equal = 1;
        less = 0;
    end else begin
        greater = 0;
        equal = 0;
        less = 1;
    end
end

endmodule
```

测试代码:

```
`timescale 1ns / 1ps

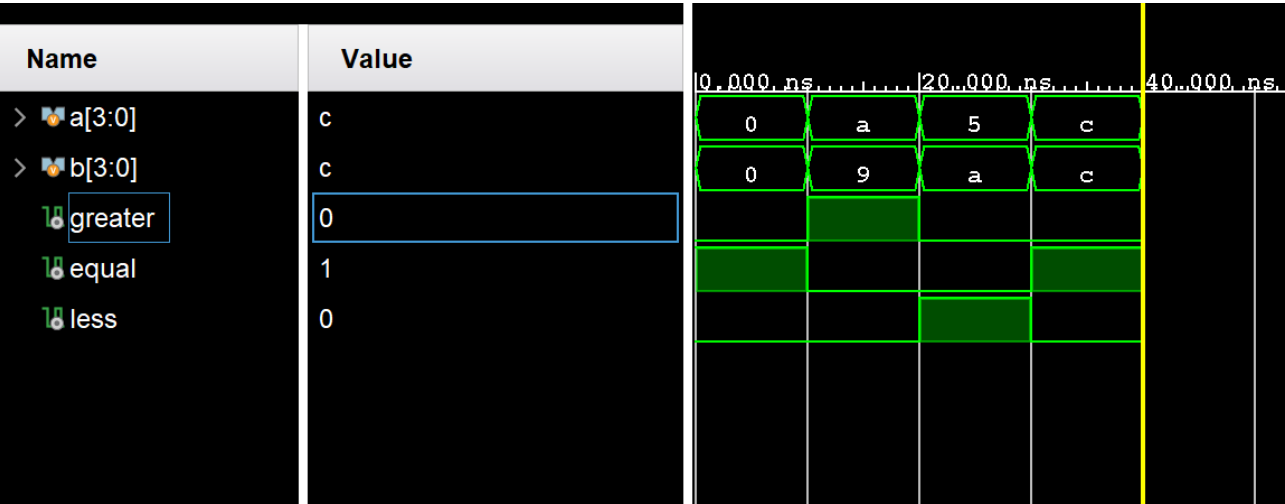
module tb_binary_comparator;

    reg [3:0] a;
    reg [3:0] b;
    wire greater;
    wire equal;
    wire less;
```

```
binary_comparator uut (  
    .a(a),  
    .b(b),  
    .greater(greater),  
    .equal(equal),  
    .less(less)  
);
```

```
initial begin  
    a = 4'b0000;  
    b = 4'b0000;  
  
    #10;  
  
    a = 4'b1010;  
    b = 4'b1001;  
    #10;  
  
    a = 4'b0111;  
    b = 4'b1010;  
    #10;  
  
    a = 4'b1000;  
    b = 4'b1000;  
    #10;  
  
    $finish;  
end  
  
endmodule
```

仿真图：



一. 练习内容

1. 用 Verilog 完成具有异步清零和异步置 1 的 D 触发器的设计,并用 Vivado 进行仿真。
2. 用 Verilog 完成时钟下降沿触发的 JK 触发器的设计, 使用 Vivado 进行仿真。

二. 源程序及仿真结果截图

1. 具有异步清零和异步置 1 的 D 触发器。

设计代码:

```
`timescale 1ns / 1ps

module d (
    input clk,
    input rst_n,
    input set_n,
    input d,
    output reg q
);

always @(posedge clk or negedge rst_n or negedge set_n) begin
    if (!rst_n) begin
        q <= 1'b0;
    end else if (!set_n) begin
        q <= 1'b1;
    end else begin
        q <= d;
    end
end

endmodule
```

测试代码:

```
`timescale 1ns / 1ps

module test;
```

```
reg clk;
reg rst_n;
reg set_n;
reg d;
wire q;

d uut (
    .clk(clk),
    .rst_n(rst_n),
    .set_n(set_n),
    .d(d),
    .q(q)
);

initial begin
    clk = 0;
    forever #5 clk = ~clk;
end
```

```
initial begin
    rst_n = 1;
    set_n = 1;
    d = 0;
    #10;

    rst_n = 0;
    #10;

    rst_n = 1;
    #10;

    set_n = 0;
    #10;

    set_n = 1;
    #10;

    d = 1;
    #100;

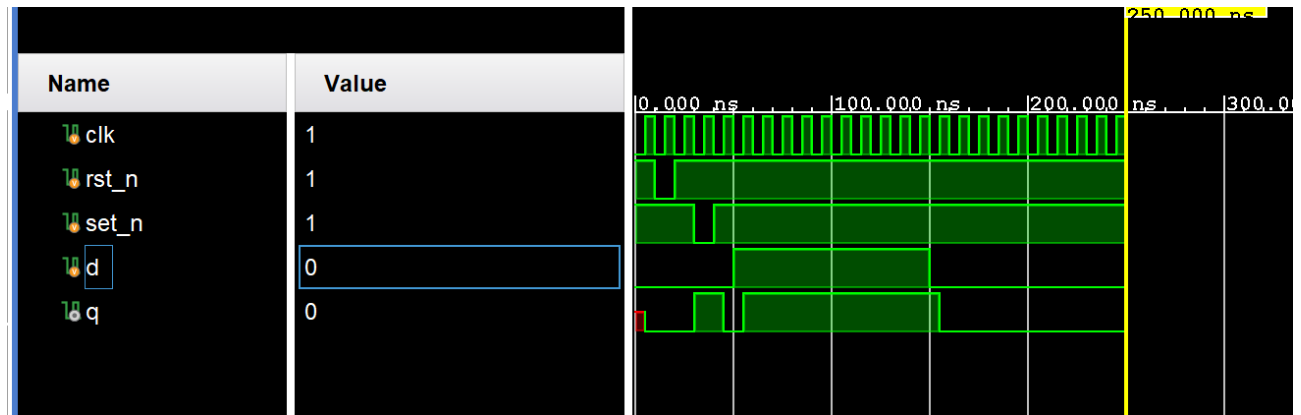
    d = 0;
    #100;

    $finish;
```

end

endmodule

仿真图：



2. 时钟下降沿触发的 JK 触发器

设计代码：

```
`timescale 1ns / 1ps
```

```
module jk (  
    input clk,  
    input rst_n,  
    input j,  
    input k,  
    output reg q,  
    output reg q_n  
);  
  
always @(negedge clk or negedge rst_n) begin  
    if (!rst_n) begin  
        q <= 1'b0;  
        q_n <= 1'b1;  
    end else begin  
        if (j == 1 && k == 1) begin  
            q <= ~q;  
            q_n <= ~q_n;  
        end else if (j == 1 && k == 0) begin  
            q <= 1'b1;  
            q_n <= 1'b0;  
        end  
    end  
end
```

```

        end else if (j == 0 && k == 1) begin
            q <= 1'b0;
            q_n <= 1'b1;
        end else begin
            end
        end
    end
end

endmodule

```

测试代码:

```
`timescale 1ns / 1ps
```

```
module test;
```

```

reg clk;
reg rst_n;
reg j;
reg k;
wire q;
wire q_n;

```

```

jk uut (
    .clk(clk),
    .rst_n(rst_n),
    .j(j),
    .k(k),
    .q(q),
    .q_n(q_n)
);

```

```

initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

```

```

initial begin
    rst_n = 1;
    j = 0;
    k = 0;
    #10;

    rst_n = 0;

```

```

#10;

rst_n = 1;
#10;

j = 1;
k = 0;
#20;

j = 0;
k = 1;
#20;

j = 1;
k = 1;
#20;

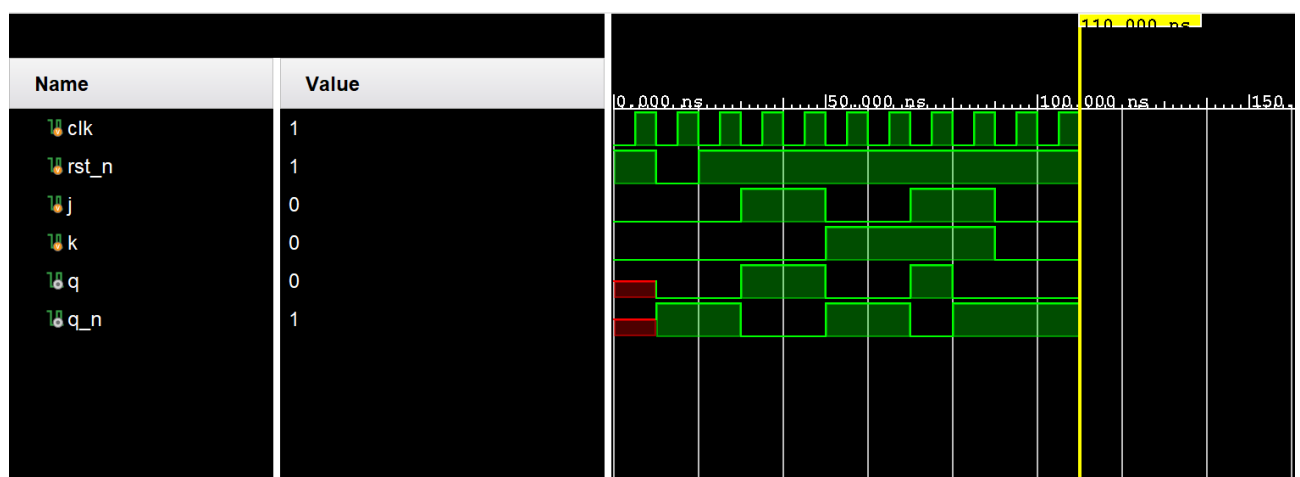
j = 0;
k = 0;
#20;

$finish;
end

endmodule

```

测试图：



一. 练习内容

1. 用 Verilog 完成具有异步清零和异步置 1 的模 10 计数器的设计，并用 Vivado 进行仿真。
2. 用 Verilog 完成具有同步清零和同步置 1 的模 16 计数器的设计，使用 Vivado 进行仿真。

二. 源程序及仿真结果截图

1. 有异步清零和异步置 1 的模 10 计数器（低电平有效）

设计代码：

```
`timescale 1ns / 1ps

module mod10_counter (
    input clk,
    input rst_n,
    input set_n,
    output reg [3:0] count
);

// Asynchronous reset and set
always @(posedge clk or negedge rst_n or negedge set_n) begin
    if (!rst_n) begin
        count <= 4'b0000;
    end else if (!set_n) begin
        count <= 4'b1001;
    end else begin
        if (count == 4'b1001) begin
            count <= 4'b0000;
        end else begin
            count <= count + 1;
        end
    end
end

endmodule
```

测试代码:

```
`timescale 1ns / 1ps
```

```
module test;
```

```
reg clk;
```

```
reg rst_n;
```

```
reg set_n;
```

```
wire [3:0] count;
```

```
mod10_counter uut (
```

```
    .clk(clk),
```

```
    .rst_n(rst_n),
```

```
    .set_n(set_n),
```

```
    .count(count)
```

```
);
```

```
initial begin
```

```
    clk = 0;
```

```
    forever #5 clk = ~clk;
```

```
end
```

```
initial begin
```

```
    rst_n = 1;
```

```
    set_n = 1;
```

```
    #10;
```

```
    rst_n = 0;
```

```
    #10;
```

```
    rst_n = 1;
```

```
    #10;
```

```
    set_n = 0;
```

```
    #10;
```

```
    set_n = 1;
```

```
    #10;
```

```
    #100;
```

```
    #50;
```

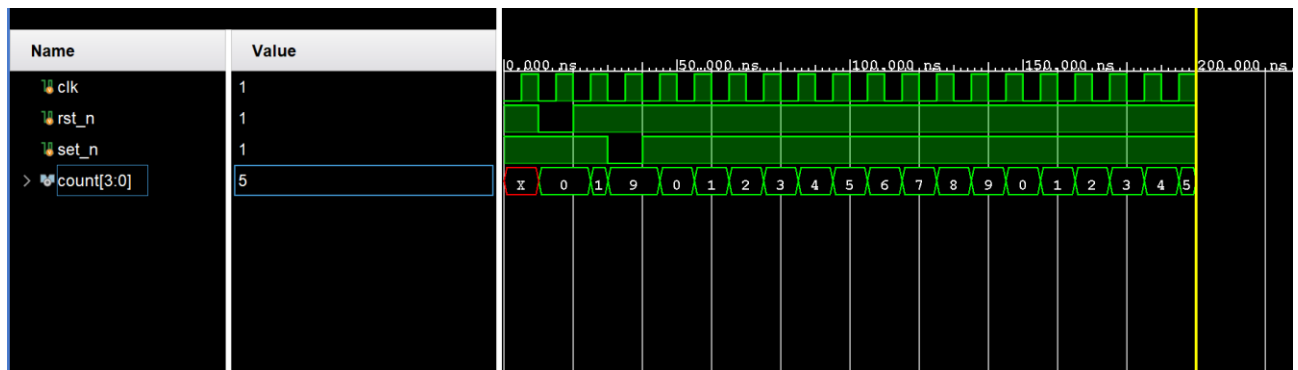
```

    $finish;
end

endmodule

```

仿真图：



2. 同步清零和同步步置 1 的模 16 计数器

设计代码：

```

`timescale 1ns / 1ps

module mod16_counter (
    input wire clk,
    input wire clr,
    input wire load,
    input wire [3:0] data,
    output reg [3:0] count
);

always @(posedge clk) begin
    if (clr) begin
        count <= 4'b0000;
    end else if (load) begin
        count <= data;
    end else begin
        if (count == 4'b1111) begin
            count <= 4'b0000;
        end else begin
            count <= count + 1;
        end
    end
end

```

```
        end  
    end  
  
endmodule
```

测试代码:

```
`timescale 1ns / 1ps  
  
module test;  
  
    reg clk;  
    reg clr;  
    reg load;  
    reg [3:0] data;  
    wire [3:0] count;  
  
    mod16_counter uut (  
        .clk(clk),  
        .clr(clr),  
        .load(load),  
        .data(data),  
        .count(count)  
    );  
  
    initial begin  
        clk = 0;  
        forever #5 clk = ~clk;  
    end  
  
    initial begin  
        clr = 0;  
        load = 0;  
        data = 4'b0000;  
  
        #10;  
  
        clr = 1;  
        #10;  
  
        clr = 0;  
        #10;  
  
        data = 4'b1010;
```

```

load = 1;
#10;

load = 0;
#10;

#100;

#50;

clr = 1;
#10;

clr = 0;
#10;

data = 4'b1100;
load = 1;
#10;

load = 0;
#10;

$finish;
end

endmodule

```

仿真图：

